

Counties

Daniel Coblentz

1/29/2025

Task 1: Start by changing the name at the top of this file to your name.

Introduction

In these activities, you will use R to look at a data file. This time, instead of “knitting,” you will click through the chunks one at a time using the green “play” (or “run”) buttons.

Packages

We will again use the **tidyverse** package, which is a toolkit for manipulating and graphing data in R. We have to load the toolkit before we can use it.

Task 2:

Type `tidyverse` (without the backticks) into the parentheses in the library command below. Then click the green “play” button at the right of the following chunk to load this package.

```
library(tidyverse)
```

Data

We will look at a data set put together by the authors of our textbook with lots of information about counties in US states. Counties are small parts of states—Maryland has 23 of them along with Baltimore city—with their own governments.

Getting data into RStudio

Task 3:

Next, we will load the data set from our textbook’s web site. The data is located at <https://www.openintro.org/data/csv/county.csv> and we will use the `read_csv` command. The arrow `<-` assigns the dataset from the link to the name “counties”.

Start typing `read_csv` in the space provided below. You’ll see possible functions appear in a tooltip. The tab key will autocomplete to the first function in the list. You can also hover over a command to see some information about how to use it, and hit F1 to get even more information in the Help window on your screen.

The `read_csv()` function can take either a file or a web address inside the parentheses, but if you use a link, you must put quotation marks around it. Try that to load the data in now. You’ll know it works if you see the number of rows and columns in the table, and some variable names.

```
county <- read_csv('https://www.openintro.org/data/csv/county.csv')
```

```
## Rows: 3142 Columns: 15
## -- Column specification -----
## Delimiter: ","
## chr (5): name, state, metro, median_edu, smoking_ban
## dbl (10): pop2000, pop2010, pop2017, pop_change, poverty, homeownership, mul...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

After you run the chunk, R gives you some information about the columns in the data set that you loaded. Most of the time, you can ignore this information, since R does the reasonable thing. If something goes wrong, though, it can be helpful.

Viewing data once you have loaded it

Task 4:

Once you have loaded data into RStudio, you can look at it by running a chunk containing its name. Type the name we gave the data into the chunk below.

```
county

## # A tibble: 3,142 x 15
##   name      state pop2000 pop2010 pop2017 pop_change poverty homeownership
##   <chr>      <chr>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>         <dbl>
## 1 Autauga County Alab~  43671   54571   55504      1.48    13.7          77.5
## 2 Baldwin County Alab~ 140415 182265 212628      9.19    11.8          76.7
## 3 Barbour County Alab~  29038  27457  25270     -6.22    27.2           68
## 4 Bibb County   Alab~  20826  22915  22668      0.73    15.2          82.9
## 5 Blount County Alab~  51024  57322  58013      0.68    15.6           82
## 6 Bullock County Alab~ 11714  10914  10309     -2.28    28.5          76.9
## 7 Butler County Alab~  21399  20947  19825     -2.69    24.4           69
## 8 Calhoun County Alab~ 112249 118572 114728     -1.51    18.6          70.7
## 9 Chambers Coun~ Alab~  36583  34215  33713     -1.2     18.8          71.4
## 10 Cherokee Coun~ Alab~  23988  25989  25857     -0.6     16.1          77.5
## # i 3,132 more rows
## # i 7 more variables: multi_unit <dbl>, unemployment_rate <dbl>, metro <chr>,
## #   median_edu <chr>, per_capita_income <dbl>, median_hh_income <dbl>,
## #   smoking_ban <chr>
```

Task 5:

To get a slightly different look, you can use the `glimpse` command. Start typing `glimpse` below and look at the tooltip in the yellow box for more information about how to use it. That may not be enough help for you, so hit F1 to get even more help, with examples, to appear. Then copy that syntax to use the `glimpse` command.

```
glimpse(county)

## Rows: 3,142
## Columns: 15
## $ name      <chr> "Autauga County", "Baldwin County", "Barbour County"~
## $ state     <chr> "Alabama", "Alabama", "Alabama", "Alabama", "Alabama~
## $ pop2000    <dbl> 43671, 140415, 29038, 20826, 51024, 11714, 21399, 11~
## $ pop2010    <dbl> 54571, 182265, 27457, 22915, 57322, 10914, 20947, 11~
## $ pop2017    <dbl> 55504, 212628, 25270, 22668, 58013, 10309, 19825, 11~
## $ pop_change <dbl> 1.48, 9.19, -6.22, 0.73, 0.68, -2.28, -2.69, -1.51, ~
```

```
## $ poverty          <dbl> 13.7, 11.8, 27.2, 15.2, 15.6, 28.5, 24.4, 18.6, 18.8~
## $ homeownership    <dbl> 77.5, 76.7, 68.0, 82.9, 82.0, 76.9, 69.0, 70.7, 71.4~
## $ multi_unit        <dbl> 7.2, 22.6, 11.1, 6.6, 3.7, 9.9, 13.7, 14.3, 8.7, 4.3~
## $ unemployment_rate <dbl> 3.86, 3.99, 5.90, 4.39, 4.02, 4.93, 5.49, 4.93, 4.08~
## $ metro             <chr> "yes", "yes", "no", "yes", "yes", "no", "no", "yes", ~
## $ median_edu        <chr> "some_college", "some_college", "hs_diploma", "hs_di~
## $ per_capita_income <dbl> 27841.70, 27779.85, 17891.73, 20572.05, 21367.39, 15~
## $ median_hh_income  <dbl> 55317, 52562, 33368, 43404, 47412, 29655, 36326, 436~
## $ smoking_ban       <chr> "none", "none", "partial", "none", "none", "none", N~
```

Task 6:

Now try the `head` function. You use it exactly the same way you used `glimpse`.

```
head(county)
```

```
## # A tibble: 6 x 15
##   name          state pop2000 pop2010 pop2017 pop_change poverty homeownership
##   <chr>         <chr>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>         <dbl>
## 1 Autauga County Alaba~  43671   54571   55504      1.48    13.7          77.5
## 2 Baldwin County Alaba~ 140415  182265  212628      9.19    11.8          76.7
## 3 Barbour County Alaba~  29038   27457   25270     -6.22    27.2           68
## 4 Bibb County    Alaba~  20826   22915   22668      0.73    15.2          82.9
## 5 Blount County  Alaba~  51024   57322   58013      0.68    15.6           82
## 6 Bullock County Alaba~  11714   10914   10309     -2.28    28.5          76.9
## # i 7 more variables: multi_unit <dbl>, unemployment_rate <dbl>, metro <chr>,
## #   median_edu <chr>, per_capita_income <dbl>, median_hh_income <dbl>,
## #   smoking_ban <chr>
```

Task 7:

Another way to look at your data is to click on the entry for the dataset in the Environment pane in the upper right.

To understand the data file, you will need to look at the “code book,” which is a document explaining what the cases and variables are. Here is a link for the code book that goes along with this data set:

<https://www.openintro.org/data/index.php?data=county>

Open the codebook in a new tab and explore what it says.

This `county` data set and its codebook are used in Sections 1.2.1 and 1.2.2 of our textbook.

Answer the questions below.

Task 8: What are the cases in this dataset?

the case are all the counties in the US from the dataset above. ## Task 9: List some of the variables in the dataset.

the dataset has 15 they are the cols in the CSV file some are ‘county_name’, ‘state_name’, ‘pop2000’, etc..

Task 10: How many cases and variables are there in this dataset in all?

15 variables, and 3142 cases in total.

Task 11: Does this dataset have any missing values? How can you tell?

```
# count missing values per column
missing_values <- county %>% summarise_all(~ sum(is.na(.)))
missing_values
```

```
## # A tibble: 1 x 15
##   name state pop2000 pop2010 pop2017 pop_change poverty homeownership
##   <int> <int>   <int>   <int>   <int>       <int>   <int>         <int>
## 1     0     0     3     0     3         3     2           0
## # i 7 more variables: multi_unit <int>, unemployment_rate <int>, metro <int>,
## #   median_edu <int>, per_capita_income <int>, median_hh_income <int>,
## #   smoking_ban <int>
```

there are a total of 603 missing values from the dataset

Data transformation examples

With R, we can easily transform data sets to get the information we want out of them. Here are some examples to get you started. Read carefully and click through the examples using the green play buttons to see what each chunk does. Then, you will perform some data transformations on your own.

1. The following chunk will let you see the counties in Maryland. The symbol %>% is called a “pipe.” It tells R to take the thing on its left and put it into the thing on its right. In this example, the command tells R, “Take the county data set and keep only the rows where the `state` variable is Maryland.”

```
county %>%
  filter(state == "Maryland")
```

```
## # A tibble: 24 x 15
##   name          state pop2000 pop2010 pop2017 pop_change poverty homeownership
##   <chr>         <chr>   <dbl>   <dbl>   <dbl>       <dbl>   <dbl>         <dbl>
## 1 Allegany Coun~ Mary~   74930   75087   71615       -2.7    16.8          70.8
## 2 Anne Arundel ~ Mary~  489656  537656  573235        3.1     6.1          75.7
## 3 Baltimore Cou~ Mary~  754292  805029  832468        1.11    9.1           67
## 4 Calvert County Mary~   74563   88737   91502        1.16    5.7           85
## 5 Caroline Coun~ Mary~   29772   33066   33193        1.61   16.5          75.8
## 6 Carroll County Mary~  150897  167134  167781        0.34    5.5          83.9
## 7 Cecil County  Mary~   85951  101108  102746        0.8     9.4          74.7
## 8 Charles County Mary~  120546  146551  159700        4.57    6.9          81.1
## 9 Dorchester Co~ Mary~   30674   32618   32162       -1.27   15.4          71.1
## 10 Frederick Cou~ Mary~  195277  233385  252022        4.42    7.2          76.8
## # i 14 more rows
## # i 7 more variables: multi_unit <dbl>, unemployment_rate <dbl>, metro <chr>,
## #   median_edu <chr>, per_capita_income <dbl>, median_hh_income <dbl>,
## #   smoking_ban <chr>
```

2. You can add more steps to the data processing pipeline. For example:

```
county %>%
  filter(state == "Maryland") %>%
  select(name, pop2017, median_hh_income)
```

```
## # A tibble: 24 x 3
##   name          pop2017 median_hh_income
##   <chr>         <dbl>         <dbl>
## 1 Allegany County   71615         42771
```

```
## 2 Anne Arundel County 573235 94502
## 3 Baltimore County 832468 71810
## 4 Calvert County 91502 100350
## 5 Caroline County 33193 52469
## 6 Carroll County 167781 90510
## 7 Cecil County 102746 70516
## 8 Charles County 159700 93973
## 9 Dorchester County 32162 50532
## 10 Frederick County 252022 88502
## # i 14 more rows
```

This chunk takes the `county` data set, keeps only the rows where the state is Maryland, and then keeps only the variables `name`, `pop2017`, and `median_hh_income`.

3. We can keep on going! The following chunk puts the counties in order of population, from smallest to largest.

```
county %>%
  filter(state == "Maryland") %>%
  select(name, pop2017, median_hh_income) %>%
  arrange(pop2017)
```

```
## # A tibble: 24 x 3
##   name                pop2017 median_hh_income
##   <chr>                <dbl>         <dbl>
## 1 Kent County          19384          56638
## 2 Somerset County     25918          39239
## 3 Garrett County      29233          48174
## 4 Dorchester County    32162          50532
## 5 Caroline County      33193          52469
## 6 Talbot County        37103          65595
## 7 Queen Anne's County 49770          89241
## 8 Worcester County     51690          59458
## 9 Allegany County      71615          42771
## 10 Calvert County      91502         100350
## # i 14 more rows
```

4. And the next chunk puts the counties in order from highest median household income to lowest median household income. (The `desc` stands for “descending order.”)

```
county %>%
  filter(state == "Maryland") %>%
  select(name, pop2017, median_hh_income) %>%
  arrange(desc(median_hh_income))
```

```
## # A tibble: 24 x 3
##   name                pop2017 median_hh_income
##   <chr>                <dbl>         <dbl>
## 1 Howard County        321113         115576
## 2 Montgomery County   1058810         103178
## 3 Calvert County       91502         100350
## 4 Anne Arundel County  573235          94502
## 5 Charles County      159700          93973
## 6 Carroll County      167781          90510
## 7 Queen Anne's County  49770          89241
## 8 Frederick County     252022          88502
## 9 St. Mary's County    112667          86508
```

```
## 10 Harford County      252160      83445
## # i 14 more rows
```

5. Here is one last example of a data transformation, which summarizes data instead of rearranging, selecting, or filtering.

```
county %>%
  count(state)
```

```
## # A tibble: 51 x 2
##   state      n
##   <chr>    <int>
## 1 Alabama      67
## 2 Alaska      29
## 3 Arizona      15
## 4 Arkansas      75
## 5 California    58
## 6 Colorado      64
## 7 Connecticut     8
## 8 Delaware       3
## 9 District of Columbia  1
## 10 Florida      67
## # i 41 more rows
```

Task 12: What does this table tell you?

The table above summarizes the dataset by counting the number of occurrences of each unique state value.

EXERCISES: Try some transformations yourself

Task 13:

Add to the following chunk to get a table of the counties in Pennsylvania.

```
county %>%
  filter(state == "Pennsylvania")
```

```
## # A tibble: 67 x 15
##   name      state pop2000 pop2010 pop2017 pop_change poverty homeownership
##   <chr>    <chr>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>         <dbl>
## 1 Adams County  Penn~   91292  101407  102336     1.26     8.8           77.8
## 2 Allegheny Cou~ Penn~ 1281666 1223348 1223048    -0.88    12.5           67
## 3 Armstrong Cou~ Penn~   72392   68941   65642    -3.26    13            76.4
## 4 Beaver County Penn~  181412  170539  166140    -2.17    10.8           75.2
## 5 Bedford County Penn~   49984   49762   48480    -1.76    13.6           79.7
## 6 Berks County  Penn~  373638  411442  417854     0.93    13.6           73.1
## 7 Blair County  Penn~  129144  127089  123457     -2      15.2           73
## 8 Bradford Coun~ Penn~   62761   62622   60853    -2.6     12.2           74.8
## 9 Bucks County  Penn~  597635  625249  628341     0.48     6.1           78.5
## 10 Butler County Penn~  174083  183862  187108     1.13     8.4           77
## # i 57 more rows
## # i 7 more variables: multi_unit <dbl>, unemployment_rate <dbl>, metro <chr>,
## #   median_edu <chr>, per_capita_income <dbl>, median_hh_income <dbl>,
## #   smoking_ban <chr>
```

Task 14:

Add to the following chunk to get a table of US counties ranked in order of population (in 2017), from smallest to largest. Keep only the variables `name`, `state`, and `pop2017`.

```
county %>%  
  select(name, state, pop2017) %>%  
  arrange(pop2017)
```

```
## # A tibble: 3,142 x 3  
##   name                state    pop2017  
##   <chr>              <chr>    <dbl>  
## 1 Kalawao County     Hawaii      88  
## 2 Loving County      Texas     134  
## 3 King County        Texas     296  
## 4 Kenedy County      Texas     417  
## 5 Arthur County      Nebraska   457  
## 6 Blaine County      Nebraska   482  
## 7 McPherson County   Nebraska   499  
## 8 Petroleum County   Montana    523  
## 9 Yakutat City and Borough Alaska    605  
## 10 Loup County        Nebraska   609  
## # i 3,132 more rows
```

Task 15:

Add to the following chunk to get a table of US counties ranked in order of median household income, from largest to smallest. Keep only the variables `name`, `state`, and `pop2017`, and `median_hh_income`.

```
county %>%  
  select(name, state, pop2017, median_hh_income) %>%  
  arrange(desc(median_hh_income))
```

```
## # A tibble: 3,142 x 4  
##   name                state    pop2017 median_hh_income  
##   <chr>              <chr>    <dbl>         <dbl>  
## 1 Loudoun County     Virginia  398080      129588  
## 2 Fairfax County     Virginia 1148433      117515  
## 3 Howard County      Maryland  321113      115576  
## 4 Falls Church city  Virginia   14583      114795  
## 5 Arlington County   Virginia  234965      112138  
## 6 Douglas County     Colorado  335299      111154  
## 7 Hunterdon County   New Jersey 125059      110969  
## 8 Los Alamos County  New Mexico  18738      110190  
## 9 Morris County      New Jersey 499693      107034  
## 10 Fairfax city       Virginia   24097      106870  
## # i 3,132 more rows
```

Task 16:

Add to the following chunk to get a table listing the number of counties in each state, ranked from largest to smallest.

```
county %>%  
  count(state) %>%  
  arrange(desc(n))
```

```
## # A tibble: 51 x 2
##   state      n
##   <chr>    <int>
## 1 Texas      254
## 2 Georgia     159
## 3 Virginia    133
## 4 Kentucky    120
## 5 Missouri    115
## 6 Kansas      105
## 7 Illinois    102
## 8 North Carolina 100
## 9 Iowa         99
## 10 Tennessee   95
## # i 41 more rows
```

When you are done, KNIT your file as a PDF and SUBMIT it in Blackboard for grading.